

Data Pipelines | HW 05

Выполнил: Васюков Александр, avvasiukov@edu.hse.ru

Github: <https://github.com/vasyukov1/hse-data-pipelines/tree/main/hw05>

Отчёт по экспериментам с оптимизацией Spark

Краткое описание

Проведены три эксперимента, направленных на оценку и оптимизацию производительности Spark-задач:

1. **Broadcast vs Shuffle-join** — проверка эффекта broadcast для малого справочника.
2. **Skew & Salting** — влияние неравномерного распределения ключей и эффект salting.
3. **Влияние repartition / coalesce на производительность** — поиск лучшего числа для количества партиций и сравнение repartition vs coalesce.

Эксперимент А — Broadcast vs обычный Shuffle-join

Проблема

Join двух таблиц (например, `orders JOIN countries`) может приводить к дорогостоящему shuffle. Для небольшого справочника (`countries`) можно избежать shuffle, разослав справочник на все исполнители (broadcast).

Причина

Shuffle — это операция, при которой Spark перемещает записи между executors так, чтобы все строки с одинаковым ключом попали в одну партицию. Такой перенос данных стоит дорого: он требует сериализации/десериализации объектов, сетевого трафика (или записи/чтения временных файлов на диск), а также синхронизации и ожидания завершения всех map-стадий перед выполнением reduce. Если одна из таблиц (справочник) маленькая и легко помещается в память исполнителя, то её имеет смысл разослать (broadcast) на все исполнители. Тогда join выполняется локально на каждой партиции — исчезает shuffle, уменьшаются сетевые и дисковые операции, падает задержка и улучшается общая пропускная способность. По этой причине в случае небольшого справочника broadcast даёт выигрыш.

Как проводился эксперимент

- Сгенерирована большая таблица `orders` и маленькая `countries`.
- Два прогона:
 1. **baseline** — обычный `orders.join(countries)`;
 2. **broadcast** — `orders.join(broadcast(countries))`.
- Замер wall-clock (`time_s`) и сбор метрики (`shuffleRead/Write`, `executorRunTime`, `numTasks`).

Результаты

Метод	time_s	shuffleRead	shuffleWrite	executorRunTime	numTasks
Baseline	1.31 s	472	472	1449 ms	25
Broadcast	0.13 s	944	944	1515 ms	50

• Числовая интерпретация

- Ускорение по wall-clock: baseline / broadcast ≈ 10.24 .
- Время уменьшилось на 10%.

Интерпретация результатов

- Ключевой показатель — wall-clock: broadcast дал очень значимое ускорение на 10%. Это ожидаемо: вместо глобальной перестановки данных (shuffle) Spark делает локальные хеш-джоины на каждой партиции, что резко сокращает сетевые/дисковые операции.
- Метрики shuffle кажутся увеличившимися при broadcast — это неприятно на первый взгляд, но в данном случае абсолютные значения малы и, вероятно, артефакт малых тестовых данных или накопленных метрик. Главное — практическая польза: время выполнения уменьшилось значительно.

Последствия

- Если справочник действительно маленький — безопасно применять broadcast().
- Надо обязательно мониторить размер справочника — broadcast с большим справочником приведёт к OOM.

Итог эксперимента

Broadcast-join — правильный инструмент для малого справочника: в ваших прогонах он дал сильную экономию времени по сравнению с обычным shuffle-join.

Эксперимент B — Skew & Salting

Проблема

Сильный skew означает, что почти все записи имеют один и тот же ключ (в нашем случае 90% записей — HOT_KEY). При shuffle-операциях (например, groupBy) все строки с этим ключом будут собраны в одну партицию/задачу. В результате одна задача становится “тяжёлой” — она обрабатывает подавляющую часть данных, занимает намного больше времени, потребляет много памяти и может начать выгружать данные на диск (spill) или вызывать долгие паузы сборщика мусора. Вся остальная часть кластера при этом ждёт завершения этой единственной долгой задачи — появляется эффект “straggler” и общая латентность задания сильно растёт. Это частая проблема распределённых вычислений: баланс нагрузки по ключам критичен для достижения хорошего времени отклика.

Причина

Shuffle распределяет данные по ключу. Если распределение ключей неравномерно, одна reducer-позиция получает непропорционально много данных.

Решение

Добавить salt к ключу, разбить “горячий” ключ на K под-ключей — сделать partial aggregation и затем агрегировать partials по оригинальному ключу. Это выравнивает нагрузку ценой дополнительного шага объединения.

Как проводился эксперимент

- Сгенерирован skewed датасет (`n_rows = 3_000_000`, `skew_fraction = 0.9`).
- Два прогона:
 1. baseline `groupBy("key")`
 2. salted approach (добавление salt, partial agg, merge).
- Собирали wall-clock и метрики.

Результаты

Метод	time_s	shuffleRead	shuffleWrite	executorRunTime	numTasks
Baseline	0.67 s	158 731	158 731	5492 ms	77
Salted (K = 10)	0.30 s	316 518	316 518	6890 ms	104

• Числовая интерпретация

- Ускорение wall-clock: `baseline / salted` ≈ 2.21 .
- Снижение времени $\approx 55\%$.
- При этом объём shuffle (read/write) увеличился примерно в 2 раза.

Интерпретация результатов

- Salting хорошо распараллелил “горячую” нагрузку: вместо одного очень длительного таска — множество более коротких, выполняемых параллельно. Это и дало более чем двукратное сокращение wall-clock.
- Trade-off: salting увеличивает суммарный объём shuffle и суммарное CPU (`executorRunTime` вырос на 1.5 ms), потому что появляется дополнительная стадия `partial-aggregate` $\rightarrow$ `merge`. Мы платим больше IO/CPU, но выигрываем в wall-clock за счёт параллелизма.

Последствия

- Salting — простое и эффективное средство борьбы с сильным перекосом, особенно когда нельзя изменить upstream распределение.
- Подбирать для salt число K эмпирически, ориентируясь на количество executors/слотов.

Итог эксперимента

Salting дал явный выигрыш по времени при явном увеличении shuffle-трафика — классическая сделка: больше работы → меньше реального времени выполнения.

Эксперимент С — Влияние repartition / coalesce на производительность

Проблема

Неправильный выбор числа партиций приводит либо к недогрузке CPU (слишком мало партиций), либо к избыточному накладному времени планировщика и I/O (слишком много партиций).

Причина

Каждая партиция — это одна задача. Слишком мало задач → не используем параллелизм. Слишком много мелких задач → scheduler overhead + мелкие shuffle-файлы → деградация.

Как проводился эксперимент

- Сгенерирован DataFrame: `n_rows, groupBy("group_id").agg(sum(value))` — намеренно создаём shuffle.
- Прогоны:
 1. baseline — исходное число партиций (`num_partitions = 8`).
 2. `repartition(p)` для набора `p`: 2, 4, 8, 16, 50, 100, 200, 500, 1000.
 3. `coalesce(p)` (для `p < baseline`): проверка быстрого уменьшения количества партиций без shuffle.
- Для каждого прогона собрали `time_s`, `shuffleReadBytes`, `shuffleWriteBytes`, `executorRunTime`, `numTasks`.

Результаты

Baseline

time_s	partitions
0.35 s	8

Repartition

time_s	partitions
0.87 s	2
0.43 s	4
0.36 s	8
0.30 s	16
0.55 s	50
1.04 s	100
1.65 s	200
3.95 s	500
7.28 s	1000

Coalesce

time_s	partitions
0.097 s	2
0.147 s	4

Числовая интерпретация

- Лучший repartition (p=16) дал ускорение $\text{baseline} / \text{best} \approx 1.187 \rightarrow$ снижение времени $\approx 16\%$.
- Coalesce(2) дал самый малый wall-clock: $\text{baseline} / \text{coalesce}(2) \approx 3.635 \rightarrow$ снижение времени $\approx 72\%$.

Интерпретация результатов

1. Поведение repartition

- При небольших p (2) задача становится менее параллельной и wall-clock растёт (0.87 s).
- С увеличением p до некоторого оптимума (p=16) wall-clock уменьшается — баланс параллелизма и overhead.
- При очень большом p (>200) wall-clock резко растёт: много мелких задач — планировщик тратит время, создаётся огромное число shuffle файлов и растёт суммарный executorRunTime. Это классический “пересорт” партиционирования.

2. Coalesce(2) оказался быстрым

- coalesce(n) пытается уменьшить число партиций без полного shuffle, поэтому сама операция repartition отсутствует и накладные расходы меньше.
- Однако shuffleReadBytes и shuffleWriteBytes при coalesce остаются огромными, также суммарный executorRunTime очень большой. Это указывает на следующее важное замечание: метрики, собранные через Spark UI в одном процессе для множества запусков, могут накапливаться и давать запутанные абсолютные значения:

- Проводили подряд много прогонов в одной сессии Spark — REST API может возвращать агрегированные/накопленные метрики по всем стадиям приложения.
- Поэтому абсолютные величины shuffleWriteBytes в ранних и поздних прогонах нельзя напрямую сравнивать без рестарта SparkSession между экспериментами.
- Тем не менее wall-clock — локальная и корректная метрика для конкретного прогона и её сравнение валидно.

3. Суммарное executorRunTime растёт с числом партиций

- executorRunTime — это суммарное CPU-время по всем задачам. При увеличении числа задач суммарное CPU часто растёт (много мелких задач → больше суммарной работы, overhead), даже если wall-clock уменьшается (за счёт параллелизма).

4. Объём shuffle растёт с числом партиций

- Repartition вызывает shuffle, и при большем числе партиций появляется множество мелких shuffle-файлов, суммарный объём traffic может вырасти (файловая и метаданные overhead). На больших кластерах такое поведение приводит к деградации сети/диска.

Последствия

- **Нельзя просто “поставить 1000 партиций — и будет быстрее”.** Есть оптимум. У нас оптимум оказался $p = 16$, но это завязано на количестве ядер и размере данных.
- **Coalesce** — удобен, когда нужно уменьшить число партиций без shuffle, но это может привести к неравномерным партициям и “горячим” таскам.

Итог эксперимента

- Есть явный “золотой” диапазон числа партиций: слишком мало — простаивает CPU; слишком много — планировщик и shuffle побеждают.
- В нашем запуске repartition(16) дал наилучший баланс среди тестов; coalesce(2) показал наименьшее wall-clock, но требует осторожной интерпретации.

Вывод

1. **Broadcast** — использовать, когда один из join-таблиц явно мал.
2. **Skew & Salting** — эффективен при сильном перекосе.
3. **Repartition / Coalesce:**
 - Возможно найти “золотую середину” партиций.
 - Не делать экстремально много партиций — scheduler overhead съест выигрыш.

- `coalesce` — быстрый способ уменьшить число партиций без `shuffle`; подходит для оптимизации небольших задач или перед записью в небольшой набор файлов.